JS Architecture

🔧 What JavaScript Provides (The Language Itself)

JavaScript (the core language defined by the **ECMAScript standard**) provides:

✅ Core Concepts:

- **Call stack**
- **Variables** (`let`, `const`, `var`)
- **Functions** (declaration, expression, arrow)
- **Loops** (`for`, `while`, etc.)
- **Conditions** (`if`, `switch`)
- **Objects, arrays, strings, numbers**
- **Promises**
- **Error handling (`try-catch`)**

But JavaScript **alone** doesn't know how to:

- wait 2 seconds (`setTimeout`)
- read files (`fs.readFile`)
- make network requests (`fetch`)
- access the DOM (`document.querySelector`)

These are provided by the **environment** where JavaScript runs.

🌍 What the Environment Provides (Browser / Node.js)

JavaScript **runs inside** an environment that adds extra **features/APIs** on top of the language.

🎯 1. **Browser APIs** (in Chrome, Firefox, etc.)

- `setTimeout`, `setInterval`
- `fetch`, `XMLHttpRequest`

- `document`, `window`, `localStorage`
- `addEventListener`
- `WebSockets`, `Geolocation`, etc.

⚙️ 2. **Node.js APIs** (when running JavaScript in Node)

- `fs.readFile`, `fs.writeFile`, `http.createServer`
- `process`, `Buffer`, `require`
- `setTimeout`, `setImmediate`, `nextTick`

> 💡 So when you say setTimeout, it's not from JavaScript itself, it's from the browser or Node.js, but JavaScript can use it.

🧠 Understanding the Call Stack, Web APIs, Callback Queue, and Event Loop

Let's tie it all together with this diagram and explanation.

🔁 Event Loop Architecture

```
Your JS Code
  |
  v
[ Call Stack ] ←——- runs your functions
    |
    | when you call setTimeout(), fetch(), etc.
    |
    v
[ Web APIs / Node APIs ]
    |     ← async ops handled here (e.g., timers, HTTP, fs)
    | after done,
    v
[ Callback Queue ]
    |
    | if stack is empty,
```

```
    v
[ Event Loop ] ⟶ pushes callback into Call Stack
```

📦 1. Call Stack

- Where JavaScript **executes** functions.
- It's **synchronous**.
- One thing at a time.

```
function sayHi() {
  console.log("Hi");
}
sayHi(); // pushed to the stack → runs → pops out
```

🌐 2. Web APIs (in browser) / Node APIs (in Node.js)

- Handle **asynchronous operations** like:
  - Timers (`setTimeout`)
  - DOM events (`onclick`)
  - AJAX (`fetch`)
  - File reading (`fs.readFile` in Node)

```
setTimeout(() => {
  console.log("Timer done");
}, 1000);
```

- `setTimeout()` is sent to **Web APIs**, which waits 1 second **outside JS engine**.

🎯 3. Callback Queue (a.k.a. Task Queue)

- Once the async operation finishes, its **callback** is put here.

```
// After 1 second
() => {
```

```
  console.log("Timer done");
}
```

🌀 4. Event Loop

- Constantly checks:

> Is the Call Stack empty?

- If yes, it pushes the first callback from **Callback Queue** into the Call Stack.

So, even though your timer ends in 1 second, it runs **only when the stack is empty**.

🔍 Let's See a Real Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Inside setTimeout");
}, 0);

console.log("End");
```

What Happens?

1. `console.log("Start")` → logs `Start`
2. `setTimeout(..., 0)` → sent to Web API (timer 0ms), callback scheduled
3. `console.log("End")` → logs `End`
4. Now stack is empty → Event loop moves callback to stack
5. `"Inside setTimeout"` is logged last

> ❗ Even with 0ms, it waits until the call stack is empty.

🧪 Another Example: fs.readFile in Node.js

```
const fs = require('fs');

fs.readFile('file.txt', 'utf8', (err, data) => {
  console.log("File read complete");
});

console.log("Reading file...");
```

Flow:

1. `fs.readFile` → Node API handles file reading **outside**
2. `console.log("Reading file...")` → printed immediately
3. After reading, callback goes to callback queue
4. Event loop picks it up, and `"File read complete"` is printed

✅ Summary Table

Concept	Provided by	Sync/Async	Purpose
`var`, `let`, `const`	JavaScript	Sync	Declare variables
`for`, `if`, `while`	JavaScript	Sync	Core language logic
`setTimeout`	Browser / Node	Async	Run code after a delay
`fs.readFile`	Node.js	Async	Read a file
`console.log`	JavaScript	Sync	Log output
Event Loop	Environment	Async	Manages async code execution
Callback Queue	Environment	Async	Holds finished async callbacks
Web APIs / Node APIs	Environment	Async	Run long tasks outside JavaScript engine